

Оптимизација на прашалници и погледи

Во оваа фаза ќе ги анализираме погледите дефинирани во [Фаза 2](#) преку прашалници базирани на реални сценарија кои ќе бидат присутни во нашата апликација и истите ќе се обидеме да ги оптимизираме.

1. Анализа на поглед 1, добивање на бројот на следбеници и бројот на профили кои ги следи даден корисник

Прашалниците кои ќе ги тестираме се следните:

```
-- 1A: информации за конкретен корисник
SELECT * FROM user_follow_info WHERE user_id = 5;

-- 1B: топ 10 најследени корисници
SELECT * FROM user_follow_info ORDER BY followers DESC LIMIT 10;
```

Време на извршување без индекси:

1A - 18.639 ms

```
Nested Loop Left Join (cost=0.84..2007.41 rows=1 width=41) (actual time=18.370..18.375 rows=1 loops=1)
-> Nested Loop Left Join (cost=0.42..1998.96 rows=1 width=24) (actual time=16.163..16.167 rows=1 loops=1)
    -> GroupAggregate (cost=0.42..8.45 rows=1 width=16) (actual time=0.099..0.101 rows=1 loops=1)
        -> Index Only Scan using follows_follower_user_id_followed_user_id_key on follows (cost=0.42..8.44 rows=1 width=16)
            Index Cond: (follower_user_id = 5)
            Heap Fetches: 1
        -> GroupAggregate (cost=0.00..1990.49 rows=1 width=16) (actual time=16.060..16.061 rows=1 loops=1)
            -> Seq Scan on follows follows_1 (cost=0.00..1986.00 rows=1793 width=8) (actual time=0.073..15.916 rows=1727 loops=1)
                Filter: (followed_user_id = 5)
                Rows Removed by Filter: 98273
            -> Index Scan using users_pkey on users u (cost=0.42..8.44 rows=1 width=25) (actual time=2.202..2.202 rows=1 loops=1)
                Index Cond: (id = 5)
Planning Time: 2.882 ms
Execution Time: 18.639 ms
```

1B - 6055.326 ms

```
Limit (cost=193416.23..193416.26 rows=10 width=41) (actual time=6013.127..6013.139 rows=10 loops=1)
-> Sort (cost=193416.23..207558.87 rows=5657054 width=41) (actual time=5995.646..5995.657 rows=10 loops=1)
    Sort Key: (COALESCE(uf2.followers, '0'::bigint)) DESC
    Sort Method: top-N heapsort Memory: 25kB
-> Hash Left Join (cost=17633.85..71169.33 rows=5657054 width=41) (actual time=1875.103..5973.746 rows=95177 loops=1)
    Hash Cond: (follows.follower_user_id = uf2.user_id)
-> Hash Right Join (cost=9034.90..60811.91 rows=92693 width=33) (actual time=1798.455..5836.973 rows=95177 loops=1)
    Hash Cond: (u.id = follows.follower_user_id)
-> Seq Scan on users u (cost=0.00..35027.00 rows=1000000 width=25) (actual time=1598.256..4992.144 rows=10000)
-> Hash (cost=7423.24..7423.24 rows=92693 width=16) (actual time=200.143..200.146 rows=95177 loops=1)
    Buckets: 16384 Batches: 16 Memory Usage: 407kB
-> GroupAggregate (cost=0.42..7423.24 rows=92693 width=16) (actual time=0.180..164.602 rows=95177 loops=1)
    Group Key: follows.follower_user_id
-> Index Only Scan using follows_follower_user_id_followed_user_id_key on follows (cost=0.42..599)
    Heap Fetches: 100000
-> Hash (cost=8386.37..8386.37 rows=12206 width=16) (actual time=76.356..76.358 rows=30563 loops=1)
    Buckets: 16384 (originally 16384) Batches: 4 (originally 2) Memory Usage: 480kB
-> Subquery Scan on uf2 (cost=7361.00..8386.37 rows=12206 width=16) (actual time=43.415..66.845 rows=30563 loops=1)
    -> HashAggregate (cost=7361.00..8264.31 rows=12206 width=16) (actual time=43.406..64.059 rows=30563 loops=1)
        Group Key: follows_1.followed_user_id
        Planned Partitions: 4 Batches: 21 Memory Usage: 601kB Disk Usage: 824kB
        -> Seq Scan on follows follows_1 (cost=0.00..1736.00 rows=100000 width=8) (actual time=0.022..11.122 rows=100000 loops=1)
Planning Time: 1.715 ms
Execution Time: 6055.326 ms
```

Веќе постои индекс на (follower_user_id, followed_user_id) поради unique constraint во ddl-от, па follower_user_id може да се земе од таму, но за да се земе followed_user_id мора да се скенира табелата секвенцијално. Затоа, го додаваме следниот индекс:

```
CREATE INDEX idx_follows_followed_user_id ON follows(followed_user_id);
```

Време на извршување со индекс:

1A - 3.993 ms (беше 18.639 ms)

```
Nested Loop Left Join (cost=27.03..806.01 rows=1 width=41) (actual time=3.702..3.704 rows=1 loops=1)
-> Nested Loop Left Join (cost=26.61..797.56 rows=1 width=24) (actual time=3.683..3.685 rows=1 loops=1)
    -> GroupAggregate (cost=0.42..8.45 rows=1 width=16) (actual time=0.097..0.098 rows=1 loops=1)
        -> Index Only Scan using follows_follower_user_id_followed_user_id_key on follows (cost=0.42..8.44 rows=1 width=16)
            Index Cond: (follower_user_id = 5)
            Heap Fetches: 1
        -> GroupAggregate (cost=26.19..789.09 rows=1 width=16) (actual time=3.582..3.583 rows=1 loops=1)
            -> Bitmap Heap Scan on follows follows_1 (cost=26.19..784.60 rows=1793 width=8) (actual time=0.363..3.396 rows=1727)
                Recheck Cond: (followed_user_id = 5)
                Heap Blocks: exact=666
            -> Bitmap Index Scan on idx_follows_followed_user_id (cost=0.00..25.74 rows=1793 width=0) (actual time=0.255..0.256 rows=1727)
                Index Cond: (followed_user_id = 5)
    -> Index Scan using users_pkey on users u (cost=0.42..8.44 rows=1 width=25) (actual time=0.016..0.016 rows=1 loops=1)
        Index Cond: (id = 5)
Planning Time: 2.325 ms
Execution Time: 3.993 ms
```

1B - 1158.681 ms (беше 6055.326 ms)

```
Limit (cost=190809.26..190809.28 rows=10 width=41) (actual time=1119.728..1119.739 rows=10 loops=1)
-> Sort (cost=190809.26..204951.89 rows=5657054 width=41) (actual time=1104.495..1104.505 rows=10 loops=1)
    Sort Key: (COALESCE(uf2.followers, '0'::bigint)) DESC
    Sort Method: top-N heapsort Memory: 25kB
-> Hash Left Join (cost=15026.87..68562.35 rows=5657054 width=41) (actual time=312.601..1083.510 rows=95177 loops=1)
    Hash Cond: (follows.follower_user_id = uf2.user_id)
-> Hash Right Join (cost=9034.90..60811.91 rows=92693 width=33) (actual time=210.510..922.405 rows=95177 loops=1)
    Hash Cond: (u.id = follows.follower_user_id)
-> Seq Scan on users u (cost=0.00..35027.00 rows=1000000 width=25) (actual time=62.089..259.973 rows=1000000)
-> Hash (cost=7423.24..7423.24 rows=92693 width=16) (actual time=148.205..148.208 rows=95177 loops=1)
    Buckets: 16384 Batches: 16 Memory Usage: 407kB
-> GroupAggregate (cost=0.42..7423.24 rows=92693 width=16) (actual time=0.140..116.621 rows=95177 loops=1)
    Group Key: follows.follower_user_id
-> Index Only Scan using follows_follower_user_id_followed_user_id_key on follows (cost=0.42..599.00 rows=92693 width=16) (actual time=0.000..0.000 rows=92693 loops=1)
    Heap Fetches: 100000
-> Hash (cost=5779.40..5779.40 rows=12206 width=16) (actual time=101.825..101.827 rows=30563 loops=1)
    Buckets: 16384 (originally 16384) Batches: 4 (originally 2) Memory Usage: 480kB
-> Subquery Scan on uf2 (cost=0.29..5779.40 rows=12206 width=16) (actual time=1.560..90.836 rows=30563 loops=1)
-> GroupAggregate (cost=0.29..5657.34 rows=12206 width=16) (actual time=1.552..87.771 rows=30563 loops=1)
    Group Key: follows_1.followed_user_id
-> Index Only Scan using idx_follows_followed_user_id on follows follows_1 (cost=0.29..5035.28 rows=12206 width=16) (actual time=0.000..0.000 rows=12206 loops=1)
    Heap Fetches: 100000
Planning Time: 1.857 ms
Execution Time: 1158.681 ms
```

Влијание на индексот врз insert/update

Тестирани прашалници:

```
INSERT INTO follows (follower_user_id, followed_user_id) VALUES (185508, 1);
```

```
UPDATE follows SET followed_user_id = 1 WHERE id = 1;
```

Без индекс - insert 1.120 ms, update 0.668 ms

```
Insert on follows (cost=0.00..0.02 rows=0 width=0) (actual time=0.332..0.332 rows=0 loops=1)
-> Result (cost=0.00..0.02 rows=1 width=32) (actual time=0.049..0.049 rows=1 loops=1)
Planning Time: 0.108 ms
Trigger for constraint follows_follower_user_id_fkey: time=0.670 calls=1
Trigger for constraint follows_followed_user_id_fkey: time=0.097 calls=1
Execution Time: 1.120 ms
```

```
Update on follows (cost=0.41..8.43 rows=0 width=0) (actual time=0.182..0.182 rows=0 loops=1)
-> Index Scan using follows_pkey on follows (cost=0.41..8.43 rows=1 width=14) (actual time=0.028..0.029 rows=1 loops=1)
    Index Cond: (id = 1)
Planning Time: 0.378 ms
Trigger for constraint follows_followed_user_id_fkey: time=0.410 calls=1
Execution Time: 0.668 ms
```

Со индекс - insert 0.541 ms, update 0.761 ms

```
Insert on follows (cost=0.00..0.02 rows=0 width=0) (actual time=0.172..0.172 rows=0 loops=1)
-> Result (cost=0.00..0.02 rows=1 width=32) (actual time=0.029..0.029 rows=1 loops=1)
Planning Time: 0.072 ms
Trigger for constraint follows_follower_user_id_fkey: time=0.326 calls=1
Trigger for constraint follows_followed_user_id_fkey: time=0.030 calls=1
Execution Time: 0.541 ms
```

```
Update on follows (cost=0.42..8.44 rows=0 width=0) (actual time=0.205..0.205 rows=0 loops=1)
-> Index Scan using follows_pkey on follows (cost=0.42..8.44 rows=1 width=14) (actual time=0.028..0.029 rows=1 loops=1)
    Index Cond: (id = 1)
Planning Time: 0.506 ms
Trigger for constraint follows_followed_user_id_fkey: time=0.472 calls=1
Execution Time: 0.761 ms
```

2. Анализа на поглед 2, најактивни корисници на платформата според бројот на слушања во изминатите 30 дена

Прашалниците кои ќе ги тестираме се следните:

```
-- 2A: активност на еден корисник
SELECT * FROM user_activity_last_30_days WHERE user_id = 100376;

-- 2B: топ 10 најактивни корисници
SELECT * FROM user_activity_last_30_days ORDER BY stream_count DESC LIMIT 10;
```

Време на извршување без индекси:

2A - 389.404 ms

```
Nested Loop (cost=1000.42..128052.52 rows=1 width=33) (actual time=341.291..351.235 rows=1 loops=1)
-> Index Scan using users_pkey on users u (cost=0.42..8.44 rows=1 width=25) (actual time=0.760..0.764 rows=1 loops=1)
    Index Cond: (id = 100376)
-> GroupAggregate (cost=1000.00..128044.07 rows=1 width=16) (actual time=340.519..350.456 rows=1 loops=1)
    -> Gather (cost=1000.00..128044.06 rows=1 width=16) (actual time=137.273..350.389 rows=2 loops=1)
        Workers Planned: 2
        Workers Launched: 2
    -> Parallel Seq Scan on song_streams ss (cost=0.00..127043.96 rows=1 width=16) (actual time=149.103..281.419 rows=1)
        Filter: ((user_id = 100376) AND (streamed_at <= now()) AND (streamed_at >= (CURRENT_DATE - 30)))
        Rows Removed by Filter: 2258560
Planning Time: 1.687 ms
Execution Time: 389.404 ms
```

2B - 1976.733 ms

```

Limit (cost=193729.83..193814.38 rows=10 width=162) (actual time=1965.655..1965.803 rows=10 loops=1)
-> Result (cost=193729.83..5188284.34 rows=590722 width=162) (actual time=1953.440..1953.585 rows=10 loops=1)
-> Sort (cost=193729.83..195206.63 rows=590722 width=16) (actual time=1953.266..1953.270 rows=10 loops=1)
    Sort Key: (count(ss.song_id)) DESC
    Sort Method: top-N heapsort  Memory: 25kB
-> HashAggregate (cost=165561.66..180964.54 rows=590722 width=16) (actual time=1326.637..1865.932 rows=593461 loops=1)
    Group Key: ss.user_id
    Planned Partitions: 8  Batches: 9  Memory Usage: 8273kB  Disk Usage: 31768kB
-> Bitmap Heap Scan on song_streams ss (cost=24927.09..103270.10 rows=972356 width=16) (actual time=109.482..5
    Recheck Cond: ((streamed_at >= (CURRENT_DATE - 30)) AND (streamed_at <= now()))
    Heap Blocks: exact=56465
-> Bitmap Index Scan on idx_song_streams_streamed_at_song_id (cost=0.00..24684.00 rows=972356 width=0) (
    Index Cond: ((streamed_at >= (CURRENT_DATE - 30)) AND (streamed_at <= now()))

SubPlan 1
-> Index Scan using users_pkey on users (cost=0.42..8.44 rows=1 width=17) (actual time=0.021..0.022 rows=1 loops=10)
    Index Cond: (id = ss.user_id)

Planning Time: 0.476 ms
JIT:
    Functions: 24
" Options: Inlining false, Optimization false, Expressions true, Deforming true"
" Timing: Generation 2.144 ms (Deform 0.808 ms), Inlining 0.000 ms, Optimization 1.505 ms, Emission 19.128 ms, Total 22.777 ms"
Execution Time: 1976.733 ms

```

Бидејќи `song_streams` нема индекс на `user_id`, за прашалник 2A потребно е секвенцијално скенирање за да се најдат стримовите за еден корисник. Затоа, додаваме индекс на таа колона:

```
CREATE INDEX idx_song_streams_user_id ON song_streams(user_id);
```

Време за извршување по додавање на индекс

2A - 0.453 ms (was 389.404 ms)

```

Nested Loop (cost=0.86..45.14 rows=1 width=33) (actual time=0.295..0.297 rows=1 loops=1)
-> Index Scan using users_pkey on users u (cost=0.42..8.44 rows=1 width=25) (actual time=0.071..0.072 rows=1 loops=1)
    Index Cond: (id = 100376)
-> GroupAggregate (cost=0.43..36.68 rows=1 width=16) (actual time=0.220..0.221 rows=1 loops=1)
    -> Index Scan using idx_song_streams_user_id on song_streams ss (cost=0.43..36.67 rows=1 width=16) (actual time=0.089..0.
        Index Cond: (user_id = 100376)
        Filter: ((streamed_at <= now()) AND (streamed_at >= (CURRENT_DATE - 30)))
        Rows Removed by Filter: 8

Planning Time: 1.843 ms
Execution Time: 0.453 ms

```

2Б остана непроменето - бидејќи прашалникот треба да направи комплексна агрегација на големи табели нема баш некој конкретен индекс што може да ги подобри перформансите. Доколку овој прашалник се извршува често во апликацијата, јасно е дека тоа може да доведе до проблеми. Ова можеме да го решиме на повеќе начини: со менување на погледот во материјализиран поглед, со кеширање и слично. Првиот пристап (материјализирани погледи) како решение ќе го погледнеме понатаму во оптимизацијата на други погледи, а конкретно за овој поглед ќе одиме со вториот пристап, поточно со кеширање кое ќе биде имплементирано во самиот апликациски код.

Влијание на индексот врз insert/update

Тестирани прашалници:

```

INSERT INTO song_streams (playback_session_id, song_id, streamed_at, user_id)
VALUES (362881, 518859, now(), 910877);

UPDATE song_streams SET user_id = 910878 WHERE id = 1;

```

Без индекс - insert 1.343 ms, update 0.372 ms

```
Insert on song_streams (cost=0.00..0.02 rows=0 width=0) (actual time=0.390..0.391 rows=0 loops=1)
-> Result (cost=0.00..0.02 rows=1 width=40) (actual time=0.056..0.057 rows=1 loops=1)
Planning Time: 0.076 ms
Trigger for constraint song_streams_playback_session_id_fkey: time=0.517 calls=1
Trigger for constraint song_streams_song_id_fkey: time=0.412 calls=1
Execution Time: 1.343 ms
```

```
Update on song_streams (cost=0.43..8.45 rows=0 width=0) (actual time=0.252..0.253 rows=0 loops=1)
-> Index Scan using song_streams_pkey on song_streams (cost=0.43..8.45 rows=1 width=14) (actual time=0.037..0.042 rows=1 loops=1)
   Index Cond: (id = 1)
Planning Time: 0.586 ms
Execution Time: 0.372 ms
```

Со индекс - insert 0.632 ms, update 0.323 ms

```
Insert on song_streams (cost=0.00..0.02 rows=0 width=0) (actual time=0.198..0.198 rows=0 loops=1)
-> Result (cost=0.00..0.02 rows=1 width=40) (actual time=0.030..0.030 rows=1 loops=1)
Planning Time: 0.049 ms
Trigger for constraint song_streams_playback_session_id_fkey: time=0.237 calls=1
Trigger for constraint song_streams_song_id_fkey: time=0.184 calls=1
Execution Time: 0.632 ms
```

```
Update on song_streams (cost=0.43..8.45 rows=0 width=0) (actual time=0.246..0.246 rows=0 loops=1)
-> Index Scan using song_streams_pkey on song_streams (cost=0.43..8.45 rows=1 width=14) (actual time=0.040..0.042 rows=1 loops=1)
   Index Cond: (id = 1)
Planning Time: 0.474 ms
Execution Time: 0.323 ms
```

3. Анализа на поглед 3, рангирање на песни по нивните просечни оценки и бројот на вкупни оценки, соодветно

Прашалниците кои ќе ги тестираме се следните:

```
-- 3A: просечна оценка за една песна
SELECT * FROM song_average_grade WHERE song_id = 1;

-- 3B: топ 10 најдобро оценети песни
SELECT * FROM song_average_grade ORDER BY avg_grade DESC, num_reviews DESC LIMIT 10;
```

Време за извршување без индекси

3A - 705.179 ms

```

Nested Loop (cost=1000.85..136433.74 rows=1 width=86) (actual time=645.720..658.140 rows=1 loops=1)
-> Nested Loop (cost=1000.43..136425.30 rows=1 width=69) (actual time=645.601..658.019 rows=1 loops=1)
    -> Index Scan using songs_pkey on songs s (cost=0.43..8.45 rows=1 width=29) (actual time=0.191..0.196 rows=1 loops=1)
        Index Cond: (id = 1)
    -> Finalize GroupAggregate (cost=1000.00..136416.84 rows=1 width=48) (actual time=645.391..657.803 rows=1 loops=1)
        -> Gather (cost=1000.00..136416.81 rows=2 width=48) (actual time=645.091..657.742 rows=3 loops=1)
            Workers Planned: 2
            Workers Launched: 2
        -> Partial GroupAggregate (cost=0.00..135416.61 rows=1 width=48) (actual time=585.860..585.861 rows=1 loops=3)
            -> Parallel Seq Scan on reviews r (cost=0.00..135416.59 rows=2 width=12) (actual time=313.794..585.721
                Filter: (song_id = 1)
                Rows Removed by Filter: 3333331
            -> Index Scan using users_pkey on users u (cost=0.42..8.44 rows=1 width=25) (actual time=0.077..0.078 rows=1 loops=1)
                Index Cond: (id = s.owner_artist_id)
Planning Time: 3.643 ms
Execution Time: 705.179 ms

```

3B - 20559.318 ms

```

Limit (cost=1696797.46..1696797.48 rows=10 width=86) (actual time=20499.331..20499.476 rows=10 loops=1)
-> Sort (cost=1696797.46..1701259.58 rows=1784848 width=86) (actual time=20067.449..20067.593 rows=10 loops=1)
    Sort Key: ag.avg_grade DESC, ag.num_reviews DESC
    Sort Method: top-N heapsort Memory: 27kB
-> Hash Join (cost=1016994.20..1658227.53 rows=1784848 width=86) (actual time=5289.583..19373.444 rows=1939589 loops=1)
    Hash Cond: (s.owner_artist_id = u.id)
-> Hash Join (cost=962631.20..1550509.28 rows=1784848 width=69) (actual time=4610.737..16796.605 rows=1939589 loops=1)
    Hash Cond: (ag.song_id = s.id)
-> Subquery Scan on ag (cost=868957.86..1407435.71 rows=1784848 width=48) (actual time=3543.588..8643.418 rows=1784848 loops=1)
    -> Finalize GroupAggregate (cost=868957.86..1389587.23 rows=1784848 width=48) (actual time=3543.580..8643.418 rows=1784848 loops=1)
        Group Key: r.song_id
        -> Gather Merge (cost=868957.86..1340503.91 rows=3569696 width=48) (actual time=3543.530..6378.421 rows=3569696 loops=1)
            Workers Planned: 2
            Workers Launched: 2
        -> Partial GroupAggregate (cost=867957.83..927472.39 rows=1784848 width=48) (actual time=3119.588..6378.421 rows=1784848 loops=1)
            Group Key: r.song_id
            -> Sort (cost=867957.83..878374.35 rows=4166608 width=12) (actual time=3194.358..3822.418 rows=4166608 loops=1)
                Sort Key: r.song_id
                Sort Method: external merge Disk: 86264kB
            -> Parallel Seq Scan on reviews r (cost=0.00..125000.08 rows=4166608 width=12) (actual time=3194.358..3822.418 rows=4166608 loops=1)
        -> Hash (cost=55943.04..55943.04 rows=1951304 width=29) (actual time=1045.998..1045.999 rows=1951232 loops=1)
            -> Seq Scan on songs s (cost=0.00..55943.04 rows=1951304 width=29) (actual time=91.198..412.261 rows=1951232 loops=1)
        -> Hash (cost=35027.00..35027.00 rows=1000000 width=25) (actual time=574.936..574.937 rows=1000000 loops=1)
            -> Seq Scan on users u (cost=0.00..35027.00 rows=1000000 width=25) (actual time=63.431..244.709 rows=1000000 loops=1)
Planning Time: 2.253 ms
Execution Time: 20559.318 ms

```

Во 3A имаме секвенцијално скенирање на `reviews` табелата за да се земат `(song_id, grade)`. Прво пробавме да додадеме индекс на `reviews(song_id)`, но планерот го игнорираше индексот бидејќи секако ќе беше потребно скенирање на табелата за да се земе `grade` колоната. Затоа можеме да воведеме сложен индекс кој ќе ги содржи сите потребни колони и ќе му овозможи на планерот да користи `Index Only Scan`.

```
CREATE INDEX idx_reviews_song_id_grade ON reviews(song_id, grade);
```

3A - 0.630 ms (was 705.179 ms)

```
Nested Loop  (cost=1.29..41.49 rows=1 width=86) (actual time=0.260..0.263 rows=1 loops=1)
  -> Nested Loop  (cost=0.86..33.05 rows=1 width=69) (actual time=0.215..0.218 rows=1 loops=1)
    -> Index Scan using songs_pkey on songs s  (cost=0.43..8.45 rows=1 width=29) (actual time=0.052..0.053 rows=1 loops=1)
        Index Cond: (id = 1)
    -> GroupAggregate  (cost=0.43..24.58 rows=1 width=48) (actual time=0.159..0.159 rows=1 loops=1)
        -> Index Only Scan using idx_reviews_song_id_grade on reviews r  (cost=0.43..24.54 rows=6 width=12) (actual time=0.100..0.101 rows=6 loops=1)
            Index Cond: (song_id = 1)
            Heap Fetches: 5
  -> Index Scan using users_pkey on users u  (cost=0.42..8.44 rows=1 width=25) (actual time=0.041..0.042 rows=1 loops=1)
      Index Cond: (id = 1)
Planning Time: 3.016 ms
Execution Time: 0.630 ms
```



Перформансите на 3B малку се подобрија (од ~20 секунди на ~14 секунди), но јасно е дека тоа е многу бавно.

Поради таа причина обичниот поглед во овој случај ќе го замениме со материјализиран поглед.

Време за извршување на прашалници по додавање на материјализиран поглед

3A - 0.19 ms

```
Index Scan using idx_sag_mv_song_id on song_average_grade_mv  (cost=0.43..8.45 rows=1 width=62) (actual time=0.074..0.075 rows=1 loops=1)
  Index Cond: (song_id = 1)
Planning Time: 1.116 ms
Execution Time: 0.189 ms
```



3B - 0.25 ms

```
Limit  (cost=0.43..1.23 rows=10 width=718) (actual time=0.081..0.209 rows=10 loops=1)
  -> Index Scan using idx_sag_mv_avg_grade on song_average_grade_mv  (cost=0.43..155226.26 rows=1939589 width=718) (actual time=0.074..0.075 rows=10 loops=1)
Planning Time: 0.936 ms
Execution Time: 0.247 ms
```



Со материјализирани погледи добиваме <1ms за читање, со тоа што свесно дозволуваме во одредени моменти на корисниците да им се прикажуваат податоци кои може да не се најновите податоци како што беше случајот кај обичните погледи.

Исто така вреди да се напомене дека во апликацискиот код ќе треба да имплементираме логика за повремено ажурирање на овие погледи, користејќи `REFRESH MATERIALIZED VIEW`, и дека еден ваков refresh трае ~45 секунди.

Влијание на индексот врз insert/update

Тестирани прашалници:

```
INSERT INTO reviews (user_id, song_id, grade, comment)
VALUES (910877, 518859, 5, 'benchmark');
```

```
UPDATE reviews SET grade = 3 WHERE id = 1;
```

Без индекс - insert 1.391 ms, update 0.268 ms

```
Insert on reviews  (cost=0.00..0.02 rows=0 width=0) (actual time=0.381..0.381 rows=0 loops=1)
  -> Result  (cost=0.00..0.02 rows=1 width=548) (actual time=0.059..0.059 rows=1 loops=1)
Planning Time: 0.126 ms
Trigger for constraint reviews_user_id_fkey: time=0.607 calls=1
Trigger for constraint reviews_song_id_fkey: time=0.380 calls=1
Execution Time: 1.391 ms
```

```
Update on reviews  (cost=0.41..8.43 rows=0 width=0) (actual time=0.148..0.148 rows=0 loops=1)
  -> Index Scan using reviews_pkey on reviews  (cost=0.41..8.43 rows=1 width=10) (actual time=0.041..0.042 rows=1 loops=1)
      Index Cond: (id = 1)
Planning Time: 0.382 ms
Execution Time: 0.268 ms
```

Со индекс - insert 0.776 ms, update 0.374 ms

```
Insert on reviews (cost=0.00..0.02 rows=0 width=0) (actual time=0.220..0.220 rows=0 loops=1)
-> Result (cost=0.00..0.02 rows=1 width=548) (actual time=0.031..0.032 rows=1 loops=1)
Planning Time: 0.111 ms
Trigger for constraint reviews_user_id_fkey: time=0.358 calls=1
Trigger for constraint reviews_song_id_fkey: time=0.183 calls=1
Execution Time: 0.776 ms
```

```
Update on reviews (cost=0.42..8.44 rows=0 width=0) (actual time=0.221..0.221 rows=0 loops=1)
-> Index Scan using reviews_pkey on reviews (cost=0.42..8.44 rows=1 width=10) (actual time=0.019..0.020 rows=1 loops=1)
    Index Cond: (id = 1)
Planning Time: 0.569 ms
Execution Time: 0.374 ms
```

4. Анализа на поглед 4, број на слушања (популарност) на артистите за изминатите 30 дена

Прашалник кој ќе го тестираме:

```
SELECT * FROM artist_popularity_last_30_days WHERE artist_display_name='Rush';
```

Време на извршување без индекси:

5295.544 ms


```

Subquery Scan on artist_popularity_last_30_days (cost=258658.87..261658.85 rows=500 width=60) (actual time=5248.277..5288.441 rows=500)
  Filter: ((artist_popularity_last_30_days.artist_display_name)::text = 'Rush'::text)
Rows Removed by Filter: 99959
-> WindowAgg (cost=258658.87..260408.85 rows=100000 width=60) (actual time=5247.481..5281.467 rows=100000 loops=1)
  -> Sort (cost=258658.85..258908.85 rows=100000 width=52) (actual time=5247.444..5253.777 rows=100000 loops=1)
    Sort Key: artist_listens.total_listens DESC
    Sort Method: quicksort Memory: 7706kB
  -> Subquery Scan on artist_listens (cost=249104.03..250354.03 rows=100000 width=52) (actual time=5163.853..5210.492 rows=100000)
    -> HashAggregate (cost=249104.03..250354.03 rows=100000 width=52) (actual time=5163.847..5200.274 rows=100000)
      Group Key: a.id
      Batches: 1 Memory Usage: 22545kB
    -> Hash Left Join (cost=173161.35..239347.87 rows=1951232 width=28) (actual time=3172.252..4602.821 rows=1951232)
      Hash Cond: (s.id = sc.song_id)
    -> Hash Right Join (cost=3618.00..64682.53 rows=1951232 width=28) (actual time=123.141..1016.733 rows=1951232)
      Hash Cond: (s.owner_artist_id = a.id)
    -> Seq Scan on songs s (cost=0.00..55942.32 rows=1951232 width=16) (actual time=70.738..304.738 rows=1951232)
    -> Hash (cost=2368.00..2368.00 rows=100000 width=20) (actual time=52.166..52.167 rows=100000)
      Buckets: 131072 Batches: 1 Memory Usage: 6157kB
    -> Seq Scan on artists a (cost=0.00..2368.00 rows=100000 width=20) (actual time=21.26..21.26 rows=100000)
  -> Hash (cost=167349.12..167349.12 rows=175538 width=16) (actual time=3048.716..3048.831 rows=30488)
    Buckets: 524288 (originally 262144) Batches: 1 (originally 1) Memory Usage: 18351kB
  -> Subquery Scan on sc (cost=163838.36..167349.12 rows=175538 width=16) (actual time=2861.6..2861.6 rows=175538)
    -> Finalize HashAggregate (cost=163838.36..165593.74 rows=175538 width=16) (actual time=2861.6..2861.6 rows=175538)
      Group Key: ss.song_id
      Batches: 1 Memory Usage: 36881kB
    -> Gather (cost=88357.02..160327.60 rows=702152 width=16) (actual time=2650.692..2650.692 rows=702152)
      Workers Planned: 4
      Workers Launched: 0
    -> Partial HashAggregate (cost=87357.02..89112.40 rows=175538 width=16) (actual time=2650.692..2650.692 rows=175538)
      Group Key: ss.song_id
      Batches: 1 Memory Usage: 36881kB
    -> Parallel Seq Scan on song_streams ss (cost=0.00..86108.61 rows=200000 width=16) (actual time=2650.692..2650.692 rows=175538)
      Filter: (streamed_at >= (CURRENT_TIMESTAMP - '30 days'::interval))
      Rows Removed by Filter: 5779244

Planning Time: 0.601 ms
JIT:
  Functions: 38
" Options: Inlining false, Optimization false, Expressions true, Deforming true"
" Timing: Generation 2.191 ms (Deform 0.875 ms), Inlining 0.000 ms, Optimization 0.746 ms, Emission 20.211 ms, Total 23.149 ms"
Execution Time: 5295.544 ms

```

Најбавните делови се секвенцијално скенирање на табелите `song_streams` и `songs`, што можеме да го оптимизираме со индекс:

```

CREATE INDEX idx_song_streams_streamed_at_song_id ON song_streams(streamed_at, song_id);
CREATE INDEX idx_songs_owner_artist_id ON songs(owner_artist_id);

```

Време на извршување со индекси:

2923.180 ms

```

WindowAgg (cost=132657.36..134407.34 rows=100000 width=60) (actual time=2878.456..2912.811 rows=100000 loops=1)
-> Sort (cost=132657.34..132907.34 rows=100000 width=52) (actual time=2878.421..2884.685 rows=100000 loops=1)
    Sort Key: artist_listens.total_listens DESC
    Sort Method: quicksort Memory: 7706kB
-> Subquery Scan on artist_listens (cost=123102.52..124352.52 rows=100000 width=52) (actual time=2795.069..2840.462 rows=1
    -> HashAggregate (cost=123102.52..124352.52 rows=100000 width=52) (actual time=2795.063..2830.225 rows=100000 loops=
        Group Key: a.id
        Batches: 1 Memory Usage: 22545kB
    -> Hash Left Join (cost=47159.84..113346.36 rows=1951232 width=28) (actual time=826.776..2239.576 rows=1953805
        Hash Cond: (s.id = sc.song_id)
    -> Hash Right Join (cost=3618.00..64682.53 rows=1951232 width=28) (actual time=118.783..1004.286 rows=19
        Hash Cond: (s.owner_artist_id = a.id)
    -> Seq Scan on songs s (cost=0.00..55942.32 rows=1951232 width=16) (actual time=69.772..303.956 ro
    -> Hash (cost=2368.00..2368.00 rows=100000 width=20) (actual time=48.780..48.781 rows=100000 loops
        Buckets: 131072 Batches: 1 Memory Usage: 6157kB
    -> Seq Scan on artists a (cost=0.00..2368.00 rows=100000 width=20) (actual time=17.634..30.8
    -> Hash (cost=41347.61..41347.61 rows=175538 width=16) (actual time=707.572..707.574 rows=304102 loops=1
        Buckets: 524288 (originally 262144) Batches: 1 (originally 1) Memory Usage: 18351kB
    -> Subquery Scan on sc (cost=37836.85..41347.61 rows=175538 width=16) (actual time=533.395..631.09
        -> HashAggregate (cost=37836.85..39592.23 rows=175538 width=16) (actual time=533.387..597.87
            Group Key: ss.song_id
            Batches: 1 Memory Usage: 36881kB
        -> Index Only Scan using idx_song_streams_streamed_at_song_id on song_streams ss (cost
            Index Cond: (streamed_at >= (CURRENT_TIMESTAMP - '30 days'::interval))
            Heap Fetches: 0

Planning Time: 0.573 ms
JIT:
    Functions: 32
" Options: Inlining false, Optimization false, Expressions true, Deforming true"
" Timing: Generation 2.147 ms (Deform 0.709 ms), Inlining 0.000 ms, Optimization 0.652 ms, Emission 16.640 ms, Total 19.439 ms"
Execution Time: 2923.180 ms

```

Сега планерот го користи креираниот индекс за табелата `song_streams`, но сепак табелата `songs` треба секвенцијално да се скенира за да се пресмета статистиката за артистите. Дополнителна оптимизација правиме со материјализиран поглед:

```

CREATE MATERIALIZED VIEW artist_popularity_last_30_days_mv AS
WITH streams_count AS (
    SELECT ss.song_id, COUNT(*) AS cnt
    FROM song_streams ss
    WHERE ss.streamed_at >= CURRENT_TIMESTAMP - INTERVAL '30 days'
    GROUP BY ss.song_id
),
artist_listens AS (
    SELECT
        a.id AS artist_id,
        a.display_name AS artist_display_name,
        COALESCE(SUM(sc.cnt), 0) AS total_listens
    FROM artists a
    LEFT JOIN songs s ON s.owner_artist_id = a.id
    LEFT JOIN streams_count sc ON sc.song_id = s.id
    GROUP BY a.id, a.display_name
)
SELECT
    ROW_NUMBER() OVER (ORDER BY total_listens DESC) AS rank,
    artist_id,
    artist_display_name,
    total_listens
FROM artist_listens;

```

Време за извршување на прашалникот по додавање на материјализиран поглед

```
Seq Scan on artist_popularity_last_30_days_mv (cost=0.00..2082.00 rows=2 width=31) (actual time=0.210..9.704 rows=41 loops=1)
  Filter: ((artist_display_name)::text = 'Rush'::text)
  Rows Removed by Filter: 99959
Planning Time: 0.094 ms
Execution Time: 9.731 ms
```

Со материјализиран поглед добиваме <10 ms за читање.

Влијание на индексот врз insert/update

Тестирани прашалници:

```
INSERT INTO song_streams (playback_session_id, song_id, streamed_at, user_id)
VALUES (362881, 518859, now(), 910877);

UPDATE song_streams SET streamed_at = streamed_at + INTERVAL '1 second' WHERE id = 1;
```

Без индекс - insert 1.575 ms, update 0.344 ms

```
Insert on song_streams (cost=0.00..0.02 rows=0 width=0) (actual time=0.391..0.391 rows=0 loops=1)
-> Result (cost=0.00..0.02 rows=1 width=40) (actual time=0.048..0.048 rows=1 loops=1)
Planning Time: 0.080 ms
Trigger for constraint song_streams_playback_session_id_fkey: time=0.555 calls=1
Trigger for constraint song_streams_song_id_fkey: time=0.607 calls=1
Execution Time: 1.575 ms
```

```
Update on song_streams (cost=0.43..8.45 rows=0 width=0) (actual time=0.230..0.231 rows=0 loops=1)
-> Index Scan using song_streams_pkey on song_streams (cost=0.43..8.45 rows=1 width=14) (actual time=0.033..0.034 rows=1 loops=1)
   Index Cond: (id = 1)
Planning Time: 0.532 ms
Execution Time: 0.344 ms
```

Со индекс - insert 0.699 ms, update 0.326 ms

```
Insert on song_streams (cost=0.00..0.02 rows=0 width=0) (actual time=0.202..0.202 rows=0 loops=1)
-> Result (cost=0.00..0.02 rows=1 width=40) (actual time=0.029..0.029 rows=1 loops=1)
Planning Time: 0.049 ms
Trigger for constraint song_streams_playback_session_id_fkey: time=0.266 calls=1
Trigger for constraint song_streams_song_id_fkey: time=0.220 calls=1
Execution Time: 0.699 ms
```

```
Update on song_streams (cost=0.43..8.45 rows=0 width=0) (actual time=0.230..0.230 rows=0 loops=1)
-> Index Scan using song_streams_pkey on song_streams (cost=0.43..8.45 rows=1 width=14) (actual time=0.033..0.034 rows=1 loops=1)
   Index Cond: (id = 1)
Planning Time: 0.647 ms
Execution Time: 0.326 ms
```

Влијание на индексот врз insert/update

Тестирани прашалници:

```
INSERT INTO songs (title, visibility, owner_artist_id, published_by_artist_id, genre)
VALUES ('benchmark', 'PUBLIC', 494, 494, 'rock');

UPDATE songs SET owner_artist_id = (owner_artist_id % 100000) + 1 WHERE id = 1;
```

Без индекс - insert 1.307 ms, update 0.938 ms

```
Insert on songs (cost=0.00..0.01 rows=0 width=0) (actual time=0.585..0.585 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=1202) (actual time=0.051..0.051 rows=1 loops=1)
Planning Time: 0.092 ms
Trigger for constraint songs_owner_artist_id_fkey: time=0.603 calls=1
Trigger for constraint songs_published_by_artist_id_fkey: time=0.071 calls=1
Trigger for constraint songs_published_by_label_id_fkey: time=0.022 calls=1
Execution Time: 1.307 ms
```

```
Update on songs (cost=0.42..8.45 rows=0 width=0) (actual time=0.376..0.376 rows=0 loops=1)
-> Index Scan using songs_pkey on songs (cost=0.42..8.45 rows=1 width=14) (actual time=0.030..0.034 rows=1 loops=1)
    Index Cond: (id = 1)
Planning Time: 0.607 ms
Trigger for constraint songs_owner_artist_id_fkey: time=0.466 calls=1
Execution Time: 0.938 ms
```

Со индекс - insert 0.627 ms, update 0.836 ms

```
Insert on songs (cost=0.00..0.01 rows=0 width=0) (actual time=0.234..0.234 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=1202) (actual time=0.022..0.022 rows=1 loops=1)
Planning Time: 0.045 ms
Trigger for constraint songs_owner_artist_id_fkey: time=0.288 calls=1
Trigger for constraint songs_published_by_artist_id_fkey: time=0.082 calls=1
Trigger for constraint songs_published_by_label_id_fkey: time=0.011 calls=1
Execution Time: 0.627 ms
```

```
Update on songs (cost=0.43..8.45 rows=0 width=0) (actual time=0.377..0.378 rows=0 loops=1)
-> Index Scan using songs_pkey on songs (cost=0.43..8.45 rows=1 width=14) (actual time=0.035..0.039 rows=1 loops=1)
    Index Cond: (id = 1)
Planning Time: 0.634 ms
Trigger for constraint songs_owner_artist_id_fkey: time=0.368 calls=1
Execution Time: 0.836 ms
```

5. Анализа на поглед 5, број на слушања (популарност) на песните за изминатите 30 дена

Прашалник кој ќе го тестираме:

```
SELECT * FROM most_popular_songs_last_30_days WHERE rank=15;
```

Време за извршување со креираниот индекс `idx_song_streams_streamed_at_song_id`

1682.017 ms

```

Subquery Scan on most_popular_songs_last_30_days (cost=96713.63..122558.33 rows=878 width=96) (actual time=1471.660..1677.911 rows=
Filter: (most_popular_songs_last_30_days.rank = 17)
Rows Removed by Filter: 16
-> WindowAgg (cost=96713.63..120364.14 rows=175535 width=96) (actual time=1471.640..1677.902 rows=17 loops=1)
    Run Condition: (row_number() OVER (?) <= 17)
-> Gather Merge (cost=96713.50..117731.12 rows=175535 width=88) (actual time=1471.591..1677.843 rows=18 loops=1)
    Workers Planned: 4
    Workers Launched: 4
-> Sort (cost=95713.44..95823.15 rows=43884 width=88) (actual time=1433.450..1433.565 rows=564 loops=5)
    Sort Key: sc.total_streams DESC
    Sort Method: quicksort Memory: 9470kB
    Worker 0: Sort Method: quicksort Memory: 9145kB
    Worker 1: Sort Method: quicksort Memory: 2511kB
    Worker 2: Sort Method: quicksort Memory: 2431kB
    Worker 3: Sort Method: quicksort Memory: 9520kB
-> Parallel Hash Join (cost=49328.85..92329.67 rows=43884 width=88) (actual time=1114.045..1407.397 rows=60789)
    Hash Cond: (s.owner_artist_id = a.id)
-> Hash Left Join (cost=46637.31..89522.94 rows=43884 width=84) (actual time=1076.395..1343.191 rows=607)
    Hash Cond: (s.published_by_label_id = la.id)
-> Hash Join (cost=43501.50..86090.08 rows=43884 width=48) (actual time=1071.308..1327.862 rows=60)
    Hash Cond: (s.id = sc.song_id)
-> Parallel Seq Scan on songs s (cost=0.00..41308.08 rows=487808 width=40) (actual time=0.09)
-> Hash (cost=41307.31..41307.31 rows=175535 width=16) (actual time=1069.687..1069.689 rows=
    Buckets: 524288 (originally 262144) Batches: 1 (originally 1) Memory Usage: 18344kB
-> Subquery Scan on sc (cost=37796.61..41307.31 rows=175535 width=16) (actual time=837)
    -> HashAggregate (cost=37796.61..39551.96 rows=175535 width=16) (actual time=837)
        Group Key: song_streams.song_id
        Batches: 1 Memory Usage: 36881kB
        Worker 0: Batches: 1 Memory Usage: 36881kB
        Worker 1: Batches: 1 Memory Usage: 36881kB
        Worker 2: Batches: 1 Memory Usage: 36881kB
        Worker 3: Batches: 1 Memory Usage: 36881kB
    -> Index Only Scan using idx_song_streams_streamed_at_song_id on song_strea
        Index Cond: (streamed_at >= (CURRENT_TIMESTAMP - '30 days'::interval))
        Heap Fetches: 95
-> Hash (cost=3131.12..3131.12 rows=375 width=48) (actual time=4.957..4.961 rows=375 loops=5)
    Buckets: 1024 Batches: 1 Memory Usage: 38kB
-> Nested Loop Left Join (cost=13.86..3131.12 rows=375 width=48) (actual time=0.389..4.675 r
    -> Hash Left Join (cost=13.44..21.18 rows=375 width=39) (actual time=0.317..0.558 rows
        Hash Cond: (la.label_id = l.id)
        -> Seq Scan on label_admins la (cost=0.00..6.75 rows=375 width=24) (actual time=
        -> Hash (cost=8.75..8.75 rows=375 width=31) (actual time=0.214..0.215 rows=375 l
            Buckets: 1024 Batches: 1 Memory Usage: 33kB
            -> Seq Scan on labels l (cost=0.00..8.75 rows=375 width=31) (actual time=0
        -> Index Scan using users_pkey on users u (cost=0.42..8.29 rows=1 width=25) (actual ti
            Index Cond: (id = la.user_id)
-> Parallel Hash (cost=1956.24..1956.24 rows=58824 width=20) (actual time=37.138..37.139 rows=20000 loop
    Buckets: 131072 Batches: 1 Memory Usage: 6528kB
-> Parallel Seq Scan on artists a (cost=0.00..1956.24 rows=58824 width=20) (actual time=27.866..30

Planning Time: 1.492 ms
JIT:
    Functions: 235
" Options: Inlining false, Optimization false, Expressions true, Deforming true"
" Timing: Generation 14.075 ms (Deform 6.062 ms), Inlining 0.000 ms, Optimization 5.950 ms, Emission 133.686 ms, Total 153.711 ms"
Execution Time: 1682.017 ms

```

За дополнителна оптимизација креираме материјализиран поглед:

```

CREATE MATERIALIZED VIEW most_popular_songs_last_30_days_mv AS
WITH stream_counts AS (
    SELECT
        song_id,
        COUNT(*) AS total_streams
    FROM song_streams
    WHERE streamed_at >= CURRENT_TIMESTAMP - INTERVAL '30 days'
    GROUP BY song_id
)
SELECT
    ROW_NUMBER() OVER (ORDER BY sc.total_streams DESC) AS rank,
    s.id AS song_id,
    s.title AS song_title,
    a.display_name AS artist_display_name,
    s.visibility AS song_visibility,
    u.username AS label_admin_username,
    l.name AS label_name,
    sc.total_streams
FROM stream_counts sc
JOIN songs s ON s.id = sc.song_id
JOIN artists a ON s.owner_artist_id = a.id
LEFT JOIN label_admins la ON s.published_by_label_id = la.id
LEFT JOIN labels l ON l.id = la.label_id
LEFT JOIN users u ON u.id = la.user_id;

```

Време за извршување на прашалникот по додавање на материјализиран поглед

```

Gather  (cost=1000.00..6167.16 rows=1 width=96) (actual time=0.381..671.251 rows=1 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  -> Parallel Seq Scan on most_popular_songs_last_30_days_mv  (cost=0.00..5167.06 rows=1 width=96) (actual time=417.275..638.156 ro
    Filter: (rank = 17)
    Rows Removed by Filter: 101315
Planning Time: 0.218 ms
Execution Time: 671.277 ms

```

Со материјализиран поглед добиваме <1s време на читање.

6. Анализа на поглед 6, детален преглед за артистите групирани по издавачка куќа на која припаѓаат

Го тестираме прашалникот:

```
SELECT * FROM label_artists_info WHERE label_name='Piercing Abyss Records';
```

Време за извршување без индекси

3197.076 ms

```

Subquery Scan on label_artists_info (cost=48955.19..49185.03 rows=1561 width=51) (actual time=3166.576..3196.926 rows=81 loops=1)
  -> GroupAggregate (cost=48955.19..49169.42 rows=1561 width=59) (actual time=3166.575..3196.914 rows=81 loops=1)
    Group Key: a.id
    -> Gather Merge (cost=48955.19..49142.10 rows=1561 width=59) (actual time=3166.542..3196.531 rows=1660 loops=1)
      Workers Planned: 4
      Workers Launched: 4
      -> Sort (cost=47955.14..47956.11 rows=390 width=59) (actual time=2548.301..2548.327 rows=332 loops=5)
        Sort Key: a.id, s.id"
        Sort Method: quicksort Memory: 68kB
        Worker 0: Sort Method: quicksort Memory: 35kB
        Worker 1: Sort Method: quicksort Memory: 39kB
        Worker 2: Sort Method: quicksort Memory: 44kB
        Worker 3: Sort Method: quicksort Memory: 60kB
        -> Nested Loop Left Join (cost=3303.17..47938.35 rows=390 width=59) (actual time=254.314..2547.203 rows=332 loops=1)
          -> Hash Join (cost=3302.88..47724.69 rows=390 width=59) (actual time=202.108..1249.756 rows=328 loops=5)
            Hash Cond: (a.id = al.artist_id)
            -> Parallel Hash Right Join (cost=2691.54..45280.17 rows=487808 width=36) (actual time=42.618..106.106 rows=487808 loops=1)
              Hash Cond: (s.owner_artist_id = a.id)
              -> Parallel Seq Scan on songs s (cost=0.00..41308.08 rows=487808 width=16) (actual time=0.37..0.37 rows=487808 loops=1)
              -> Parallel Hash (cost=1956.24..1956.24 rows=58824 width=28) (actual time=41.753..41.754 rows=58824 loops=1)
                Buckets: 131072 Batches: 1 Memory Usage: 7328kB
                -> Parallel Seq Scan on artists a (cost=0.00..1956.24 rows=58824 width=28) (actual time=0.37..0.37 rows=58824 loops=1)
            -> Hash (cost=610.34..610.34 rows=80 width=31) (actual time=152.360..152.362 rows=81 loops=5)
              Buckets: 1024 Batches: 1 Memory Usage: 14kB
              -> Hash Join (cost=9.70..610.34 rows=80 width=31) (actual time=51.681..152.281 rows=81 loops=5)
                Hash Cond: (al.label_id = l.id)
                -> Seq Scan on artist_labels al (cost=0.00..521.00 rows=30000 width=16) (actual time=1.11..1.11 rows=30000 loops=5)
                -> Hash (cost=9.69..9.69 rows=1 width=31) (actual time=33.047..33.048 rows=1 loops=5)
                  Buckets: 1024 Batches: 1 Memory Usage: 9kB
                  -> Seq Scan on labels l (cost=0.00..9.69 rows=1 width=31) (actual time=32.998..32.999 rows=1 loops=5)
                    Filter: ((name)::text = 'Piercing Abyss Records'::text)
                    Rows Removed by Filter: 374
            -> Index Scan using idx_follows_followed_user_id on follows f (cost=0.29..0.47 rows=8 width=16) (actual time=0.00..0.00 rows=8 loops=5)
              Index Cond: (followed_user_id = a.user_id)

Planning Time: 1776.191 ms
Execution Time: 3197.076 ms

```

За да го оптимизираме секвенцијалното скенирање на табелите `songs` и `artist_labels`, ги креираме индексите:

```

CREATE INDEX idx_songs_owner_artist_id ON songs(owner_artist_id);
CREATE INDEX idx_artist_labels_label_id_artist_id ON artist_labels(label_id, artist_id);

```

Време за извршување со индекси

5.213 ms

```

Subquery Scan on label_artists_info (cost=571.17..618.00 rows=1561 width=51) (actual time=4.634..5.127 rows=81 loops=1)
  -> GroupAggregate (cost=571.17..602.39 rows=1561 width=59) (actual time=4.633..5.116 rows=81 loops=1)
    Group Key: a.id
    -> Sort (cost=571.17..575.07 rows=1561 width=59) (actual time=4.614..4.708 rows=1660 loops=1)
      Sort Key: a.id, s.id"
      Sort Method: quicksort Memory: 171kB
    -> Nested Loop Left Join (cost=6.04..488.37 rows=1561 width=59) (actual time=0.125..3.388 rows=1660 loops=1)
      -> Nested Loop Left Join (cost=5.62..274.33 rows=80 width=51) (actual time=0.115..0.973 rows=83 loops=1)
        -> Nested Loop (cost=5.32..230.51 rows=80 width=51) (actual time=0.105..0.685 rows=81 loops=1)
          -> Nested Loop (cost=4.91..175.24 rows=80 width=31) (actual time=0.092..0.265 rows=81 loops=1)
            -> Seq Scan on labels l (cost=0.00..9.69 rows=1 width=31) (actual time=0.051..0.075 rows=1 loops=1)
              Filter: ((name)::text = 'Piercing Abyss Records'::text)
              Rows Removed by Filter: 374
            -> Bitmap Heap Scan on artist_labels al (cost=4.91..164.75 rows=80 width=16) (actual time=0.089..0.258 rows=81 loops=1)
              Recheck Cond: (l.id = label_id)
              Heap Blocks: exact=72
            -> Bitmap Index Scan on idx_artist_labels_label_id_artist_id (cost=0.00..4.89 rows=80 width=16) (actual time=0.089..0.258 rows=81 loops=1)
              Index Cond: (label_id = l.id)
          -> Index Scan using artists_pkey on artists a (cost=0.42..0.69 rows=1 width=28) (actual time=0.005..0.006 rows=1 loops=1)
            Index Cond: (id = al.artist_id)
        -> Index Scan using idx_follows_followed_user_id on follows f (cost=0.29..0.47 rows=8 width=16) (actual time=0.004..0.005 rows=8 loops=1)
          Index Cond: (followed_user_id = a.user_id)
      -> Index Scan using idx_songs_owner_artist_id on songs s (cost=0.43..2.42 rows=26 width=16) (actual time=0.004..0.005 rows=26 loops=1)
        Index Cond: (owner_artist_id = a.id)

Planning Time: 1.229 ms
Execution Time: 5.213 ms

```

Планерот го користи и претходно креираниот индекс `idx_follows_followed_user_id`. Индексите го забрзаа извршувањето за речиси 100%, па заклучуваме дека нема потреба од дополнителна оптимизација.

Влијание на индексот врз insert/update

Тестирани прашалници:

```

INSERT INTO songs (title, visibility, owner_artist_id, published_by_artist_id, genre)
VALUES ('benchmark', 'PUBLIC', 494, 494, 'rock');

UPDATE songs SET owner_artist_id = (owner_artist_id % 100000) + 1 WHERE id = 1;

```

Без индекс - insert 1.138 ms, update 0.883 ms

```

Insert on songs (cost=0.00..0.01 rows=0 width=0) (actual time=0.482..0.482 rows=0 loops=1)
  -> Result (cost=0.00..0.01 rows=1 width=1202) (actual time=0.044..0.045 rows=1 loops=1)
Planning Time: 0.084 ms
Trigger for constraint songs_owner_artist_id_fkey: time=0.524 calls=1
Trigger for constraint songs_published_by_artist_id_fkey: time=0.081 calls=1
Trigger for constraint songs_published_by_label_id_fkey: time=0.024 calls=1
Execution Time: 1.138 ms

```

```

Update on songs (cost=0.43..8.45 rows=0 width=0) (actual time=0.357..0.357 rows=0 loops=1)
  -> Index Scan using songs_pkey on songs (cost=0.43..8.45 rows=1 width=14) (actual time=0.032..0.035 rows=1 loops=1)
    Index Cond: (id = 1)
Planning Time: 0.481 ms
Trigger for constraint songs_owner_artist_id_fkey: time=0.431 calls=1
Execution Time: 0.883 ms

```

Со индекс - insert 0.560 ms, update 0.814 ms


```
Insert on songs (cost=0.00..0.01 rows=0 width=0) (actual time=0.205..0.205 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=1202) (actual time=0.018..0.019 rows=1 loops=1)
Planning Time: 0.043 ms
Trigger for constraint songs_owner_artist_id_fkey: time=0.240 calls=1
Trigger for constraint songs_published_by_artist_id_fkey: time=0.079 calls=1
Trigger for constraint songs_published_by_label_id_fkey: time=0.021 calls=1
Execution Time: 0.560 ms
```

```
Update on songs (cost=0.43..8.45 rows=0 width=0) (actual time=0.342..0.343 rows=0 loops=1)
-> Index Scan using songs_pkey on songs (cost=0.43..8.45 rows=1 width=14) (actual time=0.031..0.034 rows=1 loops=1)
    Index Cond: (id = 1)
Planning Time: 0.483 ms
Trigger for constraint songs_owner_artist_id_fkey: time=0.386 calls=1
Execution Time: 0.814 ms
```

Влијание на индексот врз insert/update

Тестирани прашалници:

```
INSERT INTO artist_labels (artist_id, label_id, active, start_date)
VALUES (494, 1, true, DATE '2020-01-01');

UPDATE artist_labels SET label_id = (label_id % 375) + 1 WHERE id = 1;
```

Без индекс - insert 1.219 ms, update 0.608 ms

```
Insert on artist_labels (cost=0.00..0.01 rows=0 width=0) (actual time=0.330..0.331 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=33) (actual time=0.040..0.041 rows=1 loops=1)
Planning Time: 0.082 ms
Trigger for constraint artist_labels_artist_id_fkey: time=0.601 calls=1
Trigger for constraint artist_labels_label_id_fkey: time=0.267 calls=1
Execution Time: 1.219 ms
```

```
Update on artist_labels (cost=0.41..8.44 rows=0 width=0) (actual time=0.176..0.177 rows=0 loops=1)
-> Index Scan using artist_labels_pkey on artist_labels (cost=0.41..8.44 rows=1 width=14) (actual time=0.030..0.031 rows=1 loop
    Index Cond: (id = 1)
Planning Time: 0.476 ms
Trigger for constraint artist_labels_label_id_fkey: time=0.342 calls=1
Execution Time: 0.608 ms
```



Со индекс - insert 1.334 ms, update 0.596 ms

```
Insert on artist_labels (cost=0.00..0.01 rows=0 width=0) (actual time=0.384..0.385 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=33) (actual time=0.036..0.037 rows=1 loops=1)
Planning Time: 0.078 ms
Trigger for constraint artist_labels_artist_id_fkey: time=0.655 calls=1
Trigger for constraint artist_labels_label_id_fkey: time=0.276 calls=1
Execution Time: 1.334 ms
```

```
Update on artist_labels (cost=0.41..8.44 rows=0 width=0) (actual time=0.182..0.182 rows=0 loops=1)
-> Index Scan using artist_labels_pkey on artist_labels (cost=0.41..8.44 rows=1 width=14) (actual time=0.029..0.030 rows=1 loop
    Index Cond: (id = 1)
Planning Time: 0.538 ms
Trigger for constraint artist_labels_label_id_fkey: time=0.318 calls=1
Execution Time: 0.596 ms
```



7. Анализа на поглед 7, детални информации за секоја песна

Прашалник кој го тестираме:

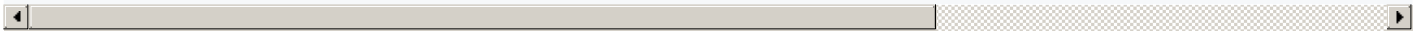
```
SELECT * FROM songs_details WHERE title='Harmony';
```

Време за извршување без индекси

93882.201 ms

```
Gather (cost=230200.46..235882.48 rows=2211 width=121) (actual time=3427.813..93875.377 rows=1683 loops=1)
  Workers Planned: 3
  Workers Launched: 3
  -> Nested Loop Left Join (cost=229200.46..234661.38 rows=713 width=121) (actual time=3393.680..41801.988 rows=421 loops=4)
    -> Merge Left Join (cost=229200.03..229252.13 rows=713 width=85) (actual time=3393.619..3396.837 rows=421 loops=4)
      Merge Cond: (s.id = pc.song_id)
      -> Sort (cost=225567.13..225568.91 rows=713 width=77) (actual time=3342.494..3342.739 rows=421 loops=4)
        Sort Key: s.id
        Sort Method: quicksort Memory: 48kB
        Worker 0: Sort Method: quicksort Memory: 44kB
        Worker 1: Sort Method: quicksort Memory: 44kB
        Worker 2: Sort Method: quicksort Memory: 72kB
      -> Nested Loop Left Join (cost=209283.95..225533.34 rows=713 width=77) (actual time=3254.123..3342.167 rows=42)
        -> Parallel Hash Right Join (cost=209283.53..225211.18 rows=713 width=72) (actual time=3254.061..3337.67)
          Hash Cond: (at.song_id = s.id)
          -> Parallel Seq Scan on album_tracks at (cost=0.00..14292.12 rows=435812 width=16) (actual time=0.
          -> Parallel Hash (cost=209276.62..209276.62 rows=553 width=64) (actual time=3253.381..3253.392 row
            Buckets: 4096 Batches: 1 Memory Usage: 192kB
            -> Hash Left Join (cost=164727.43..209276.62 rows=553 width=64) (actual time=3174.557..3252.
              Hash Cond: (s.id = sc.song_id)
              -> Hash Left Join (cost=909.90..45457.64 rows=553 width=56) (actual time=52.511..130.4
                Hash Cond: (al.label_id = l.id)
                -> Hash Left Join (cost=896.46..45442.74 rows=553 width=41) (actual time=52.297.
                  Hash Cond: (a.id = al.artist_id)
                  -> Nested Loop Left Join (cost=0.42..44542.96 rows=553 width=41) (actual t
                    -> Parallel Seq Scan on songs s (cost=0.00..42527.60 rows=553 width=
                      Filter: ((title)::text = 'Harmony'::text)
                      Rows Removed by Filter: 487388
                    -> Index Scan using artists_pkey on artists a (cost=0.42..3.64 rows=
                      Index Cond: (id = s.owner_artist_id)
                    -> Hash (cost=521.02..521.02 rows=30002 width=16) (actual time=11.075..11.
                      Buckets: 32768 Batches: 1 Memory Usage: 1663kB
                      -> Seq Scan on artist_labels al (cost=0.00..521.02 rows=30002 width=
                    -> Hash (cost=8.75..8.75 rows=375 width=31) (actual time=0.190..0.191 rows=375 1
                      Buckets: 1024 Batches: 1 Memory Usage: 33kB
                      -> Seq Scan on labels l (cost=0.00..8.75 rows=375 width=31) (actual time=0
                    -> Hash (cost=161618.58..161618.58 rows=175916 width=16) (actual time=3120.501..3120.5
                      Buckets: 1048576 (originally 262144) Batches: 1 (originally 1) Memory Usage: 380
                      -> Subquery Scan on sc (cost=158100.26..161618.58 rows=175916 width=16) (actual
                        -> HashAggregate (cost=158100.26..159859.42 rows=175916 width=16) (actual
                          Group Key: song_streams.song_id
                          Batches: 1 Memory Usage: 65553kB
                          Worker 0: Batches: 1 Memory Usage: 65553kB
                          Worker 1: Batches: 1 Memory Usage: 65553kB
                          Worker 2: Batches: 1 Memory Usage: 65553kB
                        -> Seq Scan on song_streams (cost=0.00..124221.84 rows=6775684 width
                      -> Index Scan using albums_pkey on albums alb (cost=0.42..0.45 rows=1 width=21) (actual time=0.010..0.01
                        Index Cond: (id = at.album_id)
                    -> Sort (cost=3632.91..3657.15 rows=9698 width=16) (actual time=51.065..52.559 rows=10235 loops=4)
                      Sort Key: pc.song_id
                      Sort Method: quicksort Memory: 706kB
                      Worker 0: Sort Method: quicksort Memory: 706kB
                      Worker 1: Sort Method: quicksort Memory: 706kB
                      Worker 2: Sort Method: quicksort Memory: 706kB
                      -> Subquery Scan on pc (cost=2796.77..2990.73 rows=9698 width=16) (actual time=45.054..47.748 rows=10288 loops
                        -> HashAggregate (cost=2796.77..2893.75 rows=9698 width=16) (actual time=45.048..46.739 rows=10288 loops
                          Group Key: playlist_tracks.song_id
                          Batches: 1 Memory Usage: 1425kB
                          Worker 0: Batches: 1 Memory Usage: 1425kB
```

```
Worker 0: Batches: 1 Memory Usage: 1425kB
Worker 1: Batches: 1 Memory Usage: 1425kB
Worker 2: Batches: 1 Memory Usage: 1425kB
-> Seq Scan on playlist_tracks (cost=0.00..2171.18 rows=125118 width=8) (actual time=0.038..14.065
-> Index Scan using idx_sag_mv_song_id on song_average_grade_mv sag (cost=0.43..7.58 rows=1 width=24) (actual time=91.275.
Index Cond: (song_id = s.id)
Planning Time: 2.976 ms
JIT:
Functions: 272
" Options: Inlining false, Optimization false, Expressions true, Deforming true"
" Timing: Generation 16.724 ms (Deform 7.604 ms), Inlining 0.000 ms, Optimization 6.390 ms, Emission 157.809 ms, Total 180.923 ms"
Execution Time: 93882.201 ms
```



За оптимизирање на секвенцијалните скенирање на табелите `artist_labels`, `album_tracks` и `songs` ги креираме индексите:

```
CREATE INDEX idx_songs_title
ON songs(title);

CREATE INDEX idx_album_tracks_song_id
ON album_tracks(song_id);

CREATE INDEX idx_artist_labels_artist_id
ON artist_labels(artist_id);
```

Време за извршување со индекси

3599.404 ms

```

Gather (cost=188600.68..198746.59 rows=2211 width=121) (actual time=3136.649..3592.763 rows=1683 loops=1)
  Workers Planned: 1
  Workers Launched: 1
  -> Nested Loop Left Join (cost=187600.68..197525.49 rows=1301 width=121) (actual time=3114.199..3121.952 rows=842 loops=2)
    -> Merge Left Join (cost=187600.25..187655.31 rows=1301 width=85) (actual time=3114.146..3116.186 rows=842 loops=2)
      Merge Cond: (s.id = pc.song_id)
      -> Sort (cost=183967.34..183970.60 rows=1301 width=77) (actual time=3064.947..3065.100 rows=842 loops=2)
        Sort Key: s.id
        Sort Method: quicksort Memory: 143kB
        Worker 0: Sort Method: quicksort Memory: 40kB
        -> Hash Left Join (cost=166553.22..183900.05 rows=1301 width=77) (actual time=3047.496..3064.384 rows=842 loops=2)
          Hash Cond: (s.id = sc.song_id)
          -> Nested Loop Left Join (cost=2735.67..20079.09 rows=1301 width=69) (actual time=64.333..80.679 rows=842 loops=2)
            -> Nested Loop Left Join (cost=2735.25..19491.26 rows=1301 width=64) (actual time=64.294..76.539 rows=842 loops=2)
              -> Hash Left Join (cost=2734.82..10204.79 rows=1301 width=56) (actual time=64.235..70.862 rows=842 loops=2)
                Hash Cond: (al.label_id = l.id)
                -> Nested Loop Left Join (cost=2721.39..10187.90 rows=1301 width=41) (actual time=64.000..76.539 rows=842 loops=2)
                  -> Parallel Hash Left Join (cost=2721.10..9762.21 rows=1301 width=41) (actual time=64.000..76.539 rows=842 loops=2)
                    Hash Cond: (s.owner_artist_id = a.id)
                    -> Parallel Bitmap Heap Scan on songs s (cost=29.56..7067.26 rows=1301 width=41) (actual time=0.000..1.000 rows=1301 loops=2)
                      Recheck Cond: ((title)::text = 'Harmony'::text)
                      Heap Blocks: exact=1395
                      -> Bitmap Index Scan on idx_songs_title (cost=0.00..29.01 rows=2211 width=41) (actual time=0.000..0.000 rows=1301 loops=2)
                        Index Cond: ((title)::text = 'Harmony'::text)
                    -> Parallel Hash (cost=1956.24..1956.24 rows=58824 width=20) (actual time=0.000..1.000 rows=1301 loops=2)
                      Buckets: 131072 Batches: 1 Memory Usage: 6496kB
                      -> Parallel Seq Scan on artists a (cost=0.00..1956.24 rows=58824 width=20) (actual time=0.000..1.000 rows=1301 loops=2)
                    -> Index Scan using idx_artist_labels_artist_id on artist_labels al (cost=0.29..0.29 rows=1 width=41) (actual time=0.000..0.000 rows=1 loops=2)
                      Index Cond: (artist_id = a.id)
                  -> Hash (cost=8.75..8.75 rows=375 width=31) (actual time=0.155..0.155 rows=375 loops=2)
                    Buckets: 1024 Batches: 1 Memory Usage: 33kB
                    -> Seq Scan on labels l (cost=0.00..8.75 rows=375 width=31) (actual time=0.031..0.031 rows=375 loops=2)
                  -> Index Scan using idx_album_tracks_song_id on album_tracks at (cost=0.43..7.13 rows=1 width=31) (actual time=0.000..0.000 rows=1 loops=2)
                    Index Cond: (song_id = s.id)
                -> Index Scan using albums_pkey on albums alb (cost=0.42..0.45 rows=1 width=21) (actual time=0.004..0.004 rows=1 loops=2)
                  Index Cond: (id = at.album_id)
            -> Hash (cost=161618.60..161618.60 rows=175916 width=16) (actual time=2982.071..2982.073 rows=635893 loops=2)
              Buckets: 1048576 (originally 262144) Batches: 1 (originally 1) Memory Usage: 38000kB
              -> Subquery Scan on sc (cost=158100.27..161618.60 rows=175916 width=16) (actual time=2526.994..279.000 rows=635893 loops=2)
                -> HashAggregate (cost=158100.27..159859.43 rows=175916 width=16) (actual time=2526.987..272.000 rows=635893 loops=2)
                  Group Key: song_streams.song_id
                  Batches: 1 Memory Usage: 65553kB
                  Worker 0: Batches: 1 Memory Usage: 65553kB
                  -> Seq Scan on song_streams (cost=0.00..124221.85 rows=6775685 width=8) (actual time=0.000..0.000 rows=635893 loops=2)
                -> Sort (cost=3632.91..3657.15 rows=9698 width=16) (actual time=49.155..49.933 rows=10235 loops=2)
                  Sort Key: pc.song_id
                  Sort Method: quicksort Memory: 706kB
                  Worker 0: Sort Method: quicksort Memory: 706kB
                  -> Subquery Scan on pc (cost=2796.77..2990.73 rows=9698 width=16) (actual time=43.313..46.006 rows=10288 loops=2)
                    -> HashAggregate (cost=2796.77..2893.75 rows=9698 width=16) (actual time=43.307..44.952 rows=10288 loops=2)
                      Group Key: playlist_tracks.song_id
                      Batches: 1 Memory Usage: 1425kB
                      Worker 0: Batches: 1 Memory Usage: 1425kB
                      -> Seq Scan on playlist_tracks (cost=0.00..2171.18 rows=125118 width=8) (actual time=0.039..13.544 rows=10288 loops=2)
                    -> Index Scan using idx_sag_mv_song_id on song_average_grade_mv sag (cost=0.43..7.58 rows=1 width=24) (actual time=0.006..0.006 rows=1 loops=2)
                      Index Cond: (song_id = s.id)
  Planning Time: 3.163 ms
  JIT:
    Functions: 132
    " Options: Inlining false, Optimization false, Expressions true, Deforming true"
    " Timing: Generation 7.561 ms (Deform 3.504 ms), Inlining 0.000 ms, Optimization 2.789 ms, Emission 73.213 ms, Total 83.563 ms"
  Execution Time: 3599.404 ms

```

Бидејќи сепак имаме секвенцијално скенирање на табелата `song_streams` поради групирањето на слушања по песна, дополнително оптимизираме со материјализиран погледи:

```
create materialized view song_stream_counts_mv as
select
    song_id,
    count(*) as streams
from song_streams
group by song_id;

create materialized view song_playlist_counts_mv as
select
    song_id,
    count(*) as saved_in_playlists
from playlist_tracks
group by song_id;

create or replace view song_detailed_info_view_mvs as
select
    s.title as title,
    a.display_name as artist_name,
    coalesce(l.name, 'SOLO') as label_name,
    coalesce(sc.streams, 0) as streams,
    coalesce(alb.title, 'SINGLE') as album_title,
    coalesce(pc.saved_in_playlists, 0) as saved_in_playlists,
    sag.num_reviews,
    ROUND(sag.avg_grade, 2) AS avg_grade
from songs s
left join artists a on a.id = s.owner_artist_id
left join artist_labels al on al.artist_id = a.id
left join labels l on l.id = al.label_id
left join album_tracks at on at.song_id = s.id
left join albums alb on alb.id = at.album_id
left join song_stream_counts_mv sc on sc.song_id = s.id
left join song_playlist_counts_mv pc on pc.song_id = s.id
left join song_average_grade_mv sag on sag.song_id = s.id;
```

Време за извршување на прашалникот по додавање на материјализирани погледи

333.811 ms

```

Gather (cost=25188.45..57377.30 rows=2211 width=145) (actual time=113.872..333.525 rows=1683 loops=1)
  Workers Planned: 3
  Workers Launched: 3
  -> Parallel Hash Right Join (cost=24188.45..56156.20 rows=713 width=145) (actual time=84.308..207.282 rows=421 loops=4)
    Hash Cond: (sag.song_id = s.id)
    -> Parallel Seq Scan on song_average_grade_mv sag (cost=0.00..29616.74 rows=625674 width=24) (actual time=0.016..50.181 ro
    -> Parallel Hash (cost=24176.94..24176.94 rows=921 width=85) (actual time=83.322..83.338 rows=421 loops=4)
      Buckets: 4096 Batches: 1 Memory Usage: 256kB
      -> Hash Left Join (cost=10077.11..24176.94 rows=921 width=85) (actual time=20.990..64.447 rows=421 loops=4)
        Hash Cond: (s.id = pc.song_id)
        -> Nested Loop Left Join (cost=9789.63..23885.95 rows=921 width=77) (actual time=17.300..60.507 rows=421 loops
          -> Nested Loop Left Join (cost=9789.21..23469.82 rows=921 width=72) (actual time=17.283..57.817 rows=421
            -> Hash Left Join (cost=9788.78..16895.77 rows=921 width=64) (actual time=17.245..53.588 rows=421
              Hash Cond: (al.label_id = l.id)
              -> Nested Loop Left Join (cost=9775.35..16879.89 rows=921 width=49) (actual time=17.040..53.
                -> Parallel Hash Left Join (cost=9775.06..16578.54 rows=921 width=49) (actual time=16.
                  Hash Cond: (s.owner_artist_id = a.id)
                  -> Parallel Hash Right Join (cost=7083.52..13884.59 rows=921 width=37) (actual t
                    Hash Cond: (sc.song_id = s.id)
                    -> Parallel Seq Scan on song_stream_counts_mv sc (cost=0.00..6105.55 rows=
                    -> Parallel Hash (cost=7067.26..7067.26 rows=1301 width=29) (actual time=4
                      Buckets: 4096 Batches: 1 Memory Usage: 160kB
                      -> Parallel Bitmap Heap Scan on songs s (cost=29.56..7067.26 rows=13
                        Recheck Cond: ((title)::text = 'Harmony'::text)
                        Heap Blocks: exact=455
                        -> Bitmap Index Scan on idx_songs_title (cost=0.00..29.01 rows
                          Index Cond: ((title)::text = 'Harmony'::text)
                        -> Parallel Hash (cost=1956.24..1956.24 rows=58824 width=20) (actual time=12.017
                          Buckets: 131072 Batches: 1 Memory Usage: 6528kB
                          -> Parallel Seq Scan on artists a (cost=0.00..1956.24 rows=58824 width=20)
                        -> Index Scan using idx_artist_labels_artist_id on artist_labels al (cost=0.29..0.32 r
                          Index Cond: (artist_id = a.id)
                        -> Hash (cost=8.75..8.75 rows=375 width=31) (actual time=0.179..0.180 rows=375 loops=4)
                          Buckets: 1024 Batches: 1 Memory Usage: 33kB
                          -> Seq Scan on labels l (cost=0.00..8.75 rows=375 width=31) (actual time=0.030..0.078
                        -> Index Scan using idx_album_tracks_song_id on album_tracks at (cost=0.43..7.13 rows=1 width=16)
                          Index Cond: (song_id = s.id)
                        -> Index Scan using albums_pkey on albums alb (cost=0.42..0.45 rows=1 width=21) (actual time=0.006..0.00
                          Index Cond: (id = at.album_id)
                        -> Hash (cost=158.88..158.88 rows=10288 width=16) (actual time=3.568..3.568 rows=10288 loops=4)
                          Buckets: 16384 Batches: 1 Memory Usage: 611kB
                          -> Seq Scan on song_playlist_counts_mv pc (cost=0.00..158.88 rows=10288 width=16) (actual time=0.022..1.
Planning Time: 135.345 ms
Execution Time: 333.811 ms

```

Влијание на индексот врз insert/update

Тестирани прашалници:

```

INSERT INTO songs (title, visibility, owner_artist_id, published_by_artist_id, genre)
VALUES ('benchmark', 'PUBLIC', 494, 494, 'rock');

UPDATE songs SET title = 'benchmark' WHERE id = 1;

```

Без индекс - insert 1.384 ms, update 0.487 ms

```

Insert on songs (cost=0.00..0.01 rows=0 width=0) (actual time=0.611..0.611 rows=0 loops=1)
  -> Result (cost=0.00..0.01 rows=1 width=1202) (actual time=0.041..0.042 rows=1 loops=1)
Planning Time: 0.081 ms
Trigger for constraint songs_owner_artist_id_fkey: time=0.624 calls=1
Trigger for constraint songs_published_by_artist_id_fkey: time=0.098 calls=1
Trigger for constraint songs_published_by_label_id_fkey: time=0.026 calls=1
Execution Time: 1.384 ms

```

```
Update on songs (cost=0.43..8.45 rows=0 width=0) (actual time=0.376..0.377 rows=0 loops=1)
-> Index Scan using songs_pkey on songs (cost=0.43..8.45 rows=1 width=522) (actual time=0.040..0.044 rows=1 loops=1)
    Index Cond: (id = 1)
Planning Time: 0.494 ms
Execution Time: 0.487 ms
```

Со индекс - insert 0.590 ms, update 0.426 ms

```
Insert on songs (cost=0.00..0.01 rows=0 width=0) (actual time=0.246..0.246 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=1202) (actual time=0.022..0.022 rows=1 loops=1)
Planning Time: 0.055 ms
Trigger for constraint songs_owner_artist_id_fkey: time=0.283 calls=1
Trigger for constraint songs_published_by_artist_id_fkey: time=0.036 calls=1
Trigger for constraint songs_published_by_label_id_fkey: time=0.011 calls=1
Execution Time: 0.590 ms
```

```
Update on songs (cost=0.43..8.45 rows=0 width=0) (actual time=0.345..0.345 rows=0 loops=1)
-> Index Scan using songs_pkey on songs (cost=0.43..8.45 rows=1 width=522) (actual time=0.037..0.041 rows=1 loops=1)
    Index Cond: (id = 1)
Planning Time: 0.755 ms
Execution Time: 0.426 ms
```

Влијание на индексот врз insert/update

Тестирани прашалници:

```
INSERT INTO album_tracks (album_id, song_id, track_number) VALUES (1, 1, 1);

UPDATE album_tracks SET song_id = 1 WHERE id = 1;
```

Без индекс - insert 1.416 ms, update 0.970 ms

```
Insert on album_tracks (cost=0.00..0.01 rows=0 width=0) (actual time=0.357..0.357 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=28) (actual time=0.046..0.047 rows=1 loops=1)
Planning Time: 0.082 ms
Trigger for constraint album_tracks_album_id_fkey: time=0.574 calls=1
Trigger for constraint album_tracks_song_id_fkey: time=0.462 calls=1
Execution Time: 1.416 ms
```

```
Update on album_tracks (cost=0.43..8.45 rows=0 width=0) (actual time=0.281..0.281 rows=0 loops=1)
-> Index Scan using album_tracks_pkey on album_tracks (cost=0.43..8.45 rows=1 width=14) (actual time=0.050..0.052 rows=1 loops=1)
    Index Cond: (id = 1)
Planning Time: 0.511 ms
Trigger for constraint album_tracks_song_id_fkey: time=0.574 calls=1
Execution Time: 0.970 ms
```



Со индекс - insert 0.659 ms, update 0.755 ms

```
Insert on album_tracks (cost=0.00..0.01 rows=0 width=0) (actual time=0.204..0.204 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=28) (actual time=0.021..0.022 rows=1 loops=1)
Planning Time: 0.051 ms
Trigger for constraint album_tracks_album_id_fkey: time=0.271 calls=1
Trigger for constraint album_tracks_song_id_fkey: time=0.172 calls=1
Execution Time: 0.659 ms
```

```
Update on album_tracks (cost=0.43..8.45 rows=0 width=0) (actual time=0.204..0.205 rows=0 loops=1)
-> Index Scan using album_tracks_pkey on album_tracks (cost=0.43..8.45 rows=1 width=14) (actual time=0.029..0.030 rows=1 loops=1)
    Index Cond: (id = 1)
Planning Time: 0.664 ms
Trigger for constraint album_tracks_song_id_fkey: time=0.438 calls=1
Execution Time: 0.755 ms
```

Влијание на индексот врз insert/update

Тестирани прашалници:

```
INSERT INTO artist_labels (artist_id, label_id, active, start_date)
VALUES (494, 1, true, DATE '2020-01-01');

UPDATE artist_labels SET artist_id = (artist_id % 100000) + 1 WHERE id = 1;
```

Без индекс - insert 1.106 ms, update 0.686 ms

```
Insert on artist_labels (cost=0.00..0.01 rows=0 width=0) (actual time=0.331..0.331 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=33) (actual time=0.036..0.036 rows=1 loops=1)
Planning Time: 0.090 ms
Trigger for constraint artist_labels_artist_id_fkey: time=0.539 calls=1
Trigger for constraint artist_labels_label_id_fkey: time=0.219 calls=1
Execution Time: 1.106 ms
```

```
Update on artist_labels (cost=0.41..8.44 rows=0 width=0) (actual time=0.175..0.175 rows=0 loops=1)
-> Index Scan using artist_labels_pkey on artist_labels (cost=0.41..8.44 rows=1 width=14) (actual time=0.030..0.031 rows=1 loops=1)
    Index Cond: (id = 1)
Planning Time: 0.534 ms
Trigger for constraint artist_labels_artist_id_fkey: time=0.407 calls=1
Execution Time: 0.686 ms
```

Со индекс - insert 1.287 ms, update 0.688 ms

```
Insert on artist_labels (cost=0.00..0.01 rows=0 width=0) (actual time=0.386..0.387 rows=0 loops=1)
-> Result (cost=0.00..0.01 rows=1 width=33) (actual time=0.048..0.048 rows=1 loops=1)
Planning Time: 0.148 ms
Trigger for constraint artist_labels_artist_id_fkey: time=0.615 calls=1
Trigger for constraint artist_labels_label_id_fkey: time=0.252 calls=1
Execution Time: 1.287 ms
```

```
Update on artist_labels (cost=0.41..8.44 rows=0 width=0) (actual time=0.224..0.225 rows=0 loops=1)
-> Index Scan using artist_labels_pkey on artist_labels (cost=0.41..8.44 rows=1 width=14) (actual time=0.041..0.042 rows=1 loops=1)
    Index Cond: (id = 1)
Planning Time: 0.734 ms
Trigger for constraint artist_labels_artist_id_fkey: time=0.343 calls=1
Execution Time: 0.688 ms
```

8. Анализа на поглед 8, историја на слушање песни од корисниците

Прашалник кој го тестираме:

```
SELECT *
FROM streams_history
WHERE username='adriana_klein_511'
ORDER BY streamed_at DESC;
```

Време за извршување без дополнителни индекси

0.404 ms

```
Sort (cost=52.13..52.15 rows=7 width=58) (actual time=0.333..0.335 rows=15 loops=1)
  Sort Key: ss.streamed_at DESC
  Sort Method: quicksort  Memory: 26kB
-> Nested Loop (cost=1.72..52.03 rows=7 width=58) (actual time=0.072..0.316 rows=15 loops=1)
  -> Nested Loop (cost=1.28..48.40 rows=7 width=62) (actual time=0.061..0.195 rows=15 loops=1)
    -> Nested Loop (cost=0.86..45.09 rows=7 width=49) (actual time=0.049..0.094 rows=15 loops=1)
      -> Index Scan using users_username_key on users u (cost=0.42..8.44 rows=1 width=25) (actual time=0.029..0.030 rows=1 loops=1)
        Index Cond: ((username)::text = 'adriana_klein_511'::text)
      -> Index Scan using idx_song_streams_user_id on song_streams ss (cost=0.43..36.57 rows=8 width=32) (actual time=0.030..0.061 rows=8 loops=1)
        Index Cond: (user_id = u.id)
    -> Index Scan using songs_pkey on songs s (cost=0.43..0.47 rows=1 width=21) (actual time=0.006..0.006 rows=1 loops=1)
      Index Cond: (id = ss.song_id)
  -> Index Scan using playback_sessions_pkey on playback_sessions ps (cost=0.43..0.52 rows=1 width=12) (actual time=0.007..0.007 rows=1 loops=1)
    Index Cond: (id = ss.playback_session_id)
Planning Time: 91.388 ms
Execution Time: 0.404 ms
```

Бидејќи се корситат индексите креирани за примарните клучеви, како и индексот `idx_song_streams_user_id`, нема потреба за дополнителна оптимизација на прашалникот